# Object Detection and Tracking

CodeAlpha AI Internship — Task 4

| | |
|---|---|
| **Project type** | Artificial Intelligence / Python |
| **Deliverable** | Project report with implementation source code |
| **Prepared for** | CodeAlpha AI Internship Task |

## Implementation note

This document includes a working starter implementation and the steps needed to run and test it. Update the sample content, test it in your own environment, and add your own screenshots or observations before submitting. Do not claim a test result that you have not personally verified.

# 1. Introduction

Object detection identifies objects in an image or video frame, while object tracking attempts to maintain an identity for detected objects across consecutive frames. This project uses OpenCV for video capture and YOLOv8 for object detection and tracking.

The implementation uses the pretrained YOLOv8n model and the ByteTrack tracker configuration provided by Ultralytics.

# 2. Objectives

• Read frames from a webcam or video file.
• Detect objects in each frame using a pretrained YOLO model.
• Draw class labels and bounding boxes.
• Track detected objects across frames and show tracking IDs where available.
• Display the annotated video in real time.

# 3. System requirements

Python 3.9 or later, a webcam or video file, and a computer capable of running the selected model. Install packages with **pip install ultralytics opencv-python**. The first model run may require internet access to download yolov8n.pt.

# 4. System workflow

1. Open the webcam or selected video using OpenCV.
2. Read one frame at a time.
3. Pass the frame to YOLO tracking with persist=True.
4. ByteTrack associates detections across frames.
5. The model annotates the frame with boxes, labels, and tracking IDs.
6. OpenCV displays the result until the user presses Q or the video ends.

# 5. How to run

Save the appendix code as **object_tracking.py**. Install the packages and run **python object_tracking.py**. Allow webcam access if prompted. Press Q while the video window is active to close it. To use a video file, change SOURCE from 0 to the video filename, for example **"video.mp4"**.

# 6. Testing checklist

Test with a webcam in a well-lit environment and objects clearly visible. Check whether boxes and class labels appear. Move an object through the camera view and observe whether tracking IDs remain reasonably consistent. Also test a short video file. Record actual hardware, FPS, and observations from your run.

# 7. Limitations and future enhancement

Detection quality depends on lighting, camera quality, object size, and the classes supported by the pretrained model. Tracking IDs can change when objects are hidden or leave the frame. Real-time speed depends on hardware. Improvements include saving annotated video, counting objects, selecting classes, and testing alternative trackers.

# 8. Conclusion

The project combines computer vision, pretrained deep learning, and multi-object tracking to produce annotated video with object labels and tracking identities.

# Appendix: Complete source code

```
"""
Object Detection and Tracking with YOLOv8
Install: pip install ultralytics opencv-python
Run: python object_tracking.py
Press Q in the video window to quit.
"""
import cv2
from ultralytics import YOLO

# The first run downloads the small pretrained YOLOv8n model.
model = YOLO("yolov8n.pt")

# Use 0 for the default webcam. Replace with "video.mp4" for a file.
SOURCE = 0
cap = cv2.VideoCapture(SOURCE)

if not cap.isOpened():
    raise RuntimeError("Could not open webcam/video source.")

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # persist=True keeps tracker identities consistent between frames.
    results = model.track(frame, persist=True, tracker="bytetrack.yaml",
                          conf=0.35, verbose=False)

    annotated = results[0].plot()
    cv2.imshow("Object Detection and Tracking", annotated)

    if cv2.waitKey(1) &amp;amp; 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```